

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Curso: Desarrollo Web

Catedrático: Marco Tulio Valdez Albizurez

Paw Friends

Sistema web de gestión veterinaria

Entrega final del proyecto · Lista de cotejo

Integrantes

Edgar Leonel Barco Cruz

Alan Natanihel Ruano Corvalán

Ergil Isaías Cárdenas Cruz

Enma Leticia Ramírez Aquino

Humberto Herminio Duarte Aguilar

Julio César Sandoval Benítez

Guatemala, 30 de octubre de 2026

1. Repositorio del proyecto

El código fuente completo del proyecto se encuentra en el siguiente repositorio de GitHub:

https://github.com/Edgarbarco/PAW_FRIEND

Estructura principal: `client/` (frontend en React + Vite), `server/` (backend en Node.js + Express + MongoDB), `selenium_tests/`, `k6_tests/` y `nmap_tests/` (pruebas). Las instrucciones de instalación se encuentran en el archivo `README.md` y en la documentación anexa.

2. Resumen según la rúbrica

Criterio de la rúbrica	Puntos	Cumple
Funcionalidades solicitadas completas	50	✓ Sí
Uso de tecnologías solicitadas	20	✓ Sí
Separación de conceptos Frontend-Backend	10	✓ Sí
Seguridad de aplicación mínima	10	✓ Sí
Creatividad	10	✓ Sí
Total	**100**	

3. Lista de cotejo detallada

Cada indicador señala si se cumple y dónde puede verificarse dentro del proyecto (pantalla, archivo o ruta de la API).

No.	Indicador	Sí	No	Evidencia en el proyecto
1. Funcionalidades solicitadas completas (50 pts)				
1	Registro de usuarios	✓		Pantalla de registro; `POST /api/auth/register` (`authController.js`)
2	Inicio y cierre de sesión	✓		Pantalla de login; `POST /api/auth/login`; botón Cerrar sesión
3	Gestión de usuarios (crear, editar, activar/desactivar, eliminar)	✓		Menú “Administrar Personal”; `userController.js`
4	Gestión de mascotas (CRUD)	✓		Página Mascotas; `petController.js`, `/api/pets`
5	Historial médico y vacunas por mascota	✓		Perfil de mascota; `/api/pets/:id/medical-record` y `/vaccination`
6	Gestión de citas con estados y veterinario	✓		`appointmentController.js`, `/api/appointments`
7	Agenda / calendario de citas	✓		Menú “Agenda” con calendario mensual
8	Inventario de productos (CRUD)	✓		Menú “Inventario”; `productController.js`
9	Alerta automática de stock bajo	✓		`notificationMiddleware.js` (stock ≤ 5 unidades)
10	Historial clínico de atenciones	✓		Menú “Historial de Citas”; `historyController.js`
11	Notificaciones en tiempo real	✓		Campanita; canal SSE `/api/notifications/events`
12	Notificaciones por correo electrónico	✓		`emailService.js` (Nodemailer)
13	Formulario de contacto y administración de mensajes	✓		Menú “Mensajes”; `/api/contacto`
14	Dashboard con resumen y métricas	✓		Pantalla de inicio; `metricsController.js`
15	Pantallas de vacunación, peluquería y hospitalización	✓		Menú lateral del dashboard
2. Uso de tecnologías solicitadas (20 pts)				
16	Base de datos MongoDB	✓		Modelos con Mongoose en `server/models`
17	Backend con Node.js y Express	✓		`server/index.js`, rutas en `server/routes`
18	Frontend con React	✓		`client/src` (componentes, páginas, contextos)
19	API REST con intercambio en JSON	✓		Rutas `/api/...` documentadas

No.	Indicador	Sí	No	Evidencia en el proyecto
20	Herramientas de apoyo: Vite, Ant Design, Git/GitHub	✓		`client/package.json`; repositorio en GitHub
3. Separación de conceptos Frontend-Backend (10 pts)				
21	Proyectos separados para cliente y servidor	✓		Carpetas `client/` y `server/`, cada una con su `package.json`
22	Comunicación únicamente por API HTTP	✓		El cliente consume la API mediante `VITE_API_URL`
23	Backend organizado por capas (MVC)	✓		`models/`, `controllers/`, `routes/`, `middlewares/`, `services/`
24	Frontend organizado por componentes	✓		`components/`, `pages/`, `contexts/`, `hooks/`
4. Seguridad de aplicación mínima (10 pts)				
25	Contraseñas encriptadas	✓		`bcrypt.hash(password, 12)` en el registro
26	Autenticación con token JWT	✓		`jsonwebtoken`; middleware `protect`
27	Control de acceso por roles	✓		Middleware `authorize('admin')`; máximo 3 administradores
28	Rutas protegidas en el frontend	✓		`ProtectedRoute.jsx` redirige al login sin sesión
29	Credenciales fuera del código	✓		Archivo `.env` y `.gitignore`
30	Manejo centralizado de errores	✓		Manejador global en `index.js` y clase `AppError`
5. Creatividad (10 pts)				
31	Notificaciones en tiempo real (Server-Sent Events)	✓		Avisos instantáneos sin recargar la página
32	Envío de correos con plantillas HTML	✓		`emailService.js`
33	Diseño visual propio con Ant Design	✓		Pantallas de inicio, login y dashboard
34	Pruebas automatizadas: Jest, Selenium, k6 y nmap	✓		`server/__tests__` (37/37 aprobadas), `selenium_tests/`, `k6_tests/`, `nmap_tests/`
35	Documentación técnica completa	✓		Documento anexo con arquitectura, API y capturas

Observación: el proyecto incluye además una documentación técnica completa (arquitectura, módulos, API, base de datos, seguridad, pruebas, instalación y distribución del trabajo) con capturas de pantalla del sistema en funcionamiento.



UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Curso: Desarrollo Web

Catedrático: Marco Tulio Valdez Albizurez

Paw Friends

Sistema web de gestión veterinaria

Documentación técnica del proyecto

Integrantes

Edgar Leonel Barco Cruz

Alan Natanihel Ruano Corvalán

Ergil Isaías Cárdenas Cruz

Enma Leticia Ramírez Aquino

Humberto Herminio Duarte Aguilar

Julio César Sandoval Benítez

Guatemala, 30 de octubre de 2026

Índice

1. Introducción 4
1.1 Descripción del proyecto 4
1.2 Problema que resuelve 4
1.3 Objetivos 4
1.4 Alcance 4
2. Tecnologías utilizadas 6
3. Arquitectura del sistema 7
3.1 Arquitectura general (MERN) 7
3.2 Patrón MVC en el backend 7
3.3 Estructura de carpetas 7
3.4 Punto de entrada del servidor (index.js) 8
4. Módulos del sistema 9
4.1 Autenticación y usuarios 9
4.2 Mascotas 10
4.3 Citas y agenda 11
4.4 Inventario 12
4.5 Historial clínico 13
4.6 Notificaciones 14
4.7 Métricas y dashboard 15
4.8 Contactos 16
5. API REST 18
6. Base de datos 20
7. Seguridad 21
8. Pruebas y control de calidad 22
9. Instalación y ejecución 23
9.1 Requisitos 23
9.2 Pasos 23
10. Distribución del trabajo 24
10.1 Edgar Leonel Barco Cruz — Líder del proyecto 24
10.2 Alan Natanihel Ruano Corvalán — Configuración base 24
10.3 Ergil Isaías Cárdenas Cruz — Estructura del frontend 24
10.4 Enma Leticia Ramírez Aquino — Módulos principales 25
10.5 Humberto Herminio Duarte Aguilar — Notificaciones y métricas 25

10.6 Julio César Sandoval Benítez — Dashboard y contactos 25
11. Conclusiones 26
Anexo A. Capturas adicionales 27

1. Introducción

1.1 Descripción del proyecto

Paw Friends es un sistema web para la gestión integral de una clínica veterinaria. Reúne en una sola plataforma el registro de mascotas y su historial médico, la agenda de citas, el control del inventario, la administración del personal, las notificaciones y un panel de métricas para la toma de decisiones.

El sistema fue desarrollado con el stack **MERN** (MongoDB, Express, React y Node.js), separando el proyecto en dos aplicaciones: un **cliente** (frontend) que se ejecuta en el navegador y un **servidor** (backend) que expone una API REST y se comunica con la base de datos.

1.2 Problema que resuelve

En muchas veterinarias la información se maneja en papel, hojas de cálculo o sistemas aislados: las citas se anotan en agendas, el inventario se revisa a mano y el historial de cada mascota queda disperso. Esto provoca pérdida de información, citas duplicadas, productos agotados sin aviso y poca visibilidad del funcionamiento de la clínica.

Paw Friends centraliza estos procesos, controla el acceso según el rol de cada usuario y avisa automáticamente de eventos importantes, como un producto con stock bajo o una nueva cita.

1.3 Objetivos

Objetivo general

Desarrollar una aplicación web que permita administrar de forma centralizada, segura y eficiente las operaciones principales de una clínica veterinaria.

Objetivos específicos

- Implementar un sistema de autenticación seguro con contraseñas encriptadas, tokens JWT y control de acceso por roles.
- Gestionar mascotas con su historial médico y registro de vacunas.
- Administrar citas con calendario, estados y veterinario asignado.
- Controlar el inventario de productos con alertas automáticas de stock bajo.
- Notificar en tiempo real y por correo electrónico los eventos relevantes.
- Presentar métricas de la clínica en un dashboard.
- Validar el sistema con pruebas unitarias, de interfaz, de carga y de seguridad.

1.4 Alcance

El sistema incluye: registro e inicio de sesión, gestión de usuarios (solo administradores), mascotas, citas, inventario, historial clínico, formulario de contacto, notificaciones (tiempo real y correo), métricas y pantallas de agenda, vacunación, peluquería y hospitalización.

Quedan fuera del alcance de esta versión: pagos en línea, facturación electrónica, aplicación móvil nativa y despliegue en producción.

2. Tecnologías utilizadas

La siguiente tabla resume las tecnologías y librerías principales, y el propósito de cada una dentro del proyecto.

Capa	Tecnología	Uso en el proyecto
Frontend	React 18	Construcción de la interfaz mediante componentes.
Frontend	Vite 4	Servidor de desarrollo y empaquetado del cliente.
Frontend	Ant Design 5	Componentes visuales: tablas, formularios, calendario, modales.
Frontend	React Router 6	Navegación entre páginas y rutas protegidas.
Frontend	Axios / Fetch	Peticiones HTTP a la API.
Backend	Node.js	Entorno de ejecución de JavaScript en el servidor.
Backend	Express 4	Framework para la API REST, rutas y middlewares.
Backend	Mongoose 8	Modelado de datos y conexión con MongoDB.
Backend	jsonwebtoken (JWT)	Generación y verificación de tokens de sesión.
Backend	bcryptjs	Encriptación (hash) de contraseñas.
Backend	Nodemailer	Envío de notificaciones por correo electrónico.
Backend	dotenv / cors	Variables de entorno y permisos entre orígenes.
Base de datos	MongoDB	Base de datos NoSQL orientada a documentos.
Pruebas	Jest	Pruebas unitarias del servidor.
Pruebas	Selenium (Python)	Pruebas end-to-end sobre la interfaz.
Pruebas	k6	Pruebas de carga al inicio de sesión.
Pruebas	nmap	Escaneo de puertos y seguridad.
Herramientas	Git / GitHub, ESLint, nodemon	Control de versiones, calidad de código y recarga automática.

3. Arquitectura del sistema

3.1 Arquitectura general (MERN)

El sistema sigue una arquitectura **cliente-servidor** de tres capas. El cliente (React) muestra la interfaz y envía peticiones HTTP en formato JSON; el servidor (Express) valida la petición, aplica la lógica de negocio y consulta la base de datos (MongoDB) mediante Mongoose; finalmente responde en JSON y el cliente actualiza la pantalla.

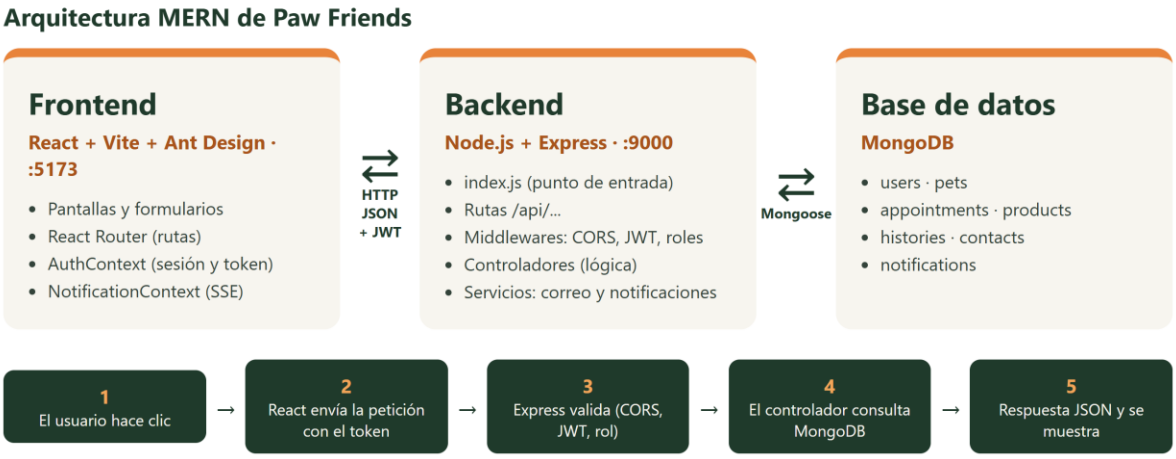


Figura 1. Arquitectura del sistema y flujo de una petición.

3.2 Patrón MVC en el backend

El servidor está organizado según el patrón **Modelo–Vista–Controlador**, donde la "vista" es el frontend de React:

- **Modelos** (`server/models`): definen la estructura de cada colección con esquemas de Mongoose.
- **Controladores** (`server/controllers`): contienen la lógica de cada operación (crear, listar, actualizar, eliminar).
- **Rutas** (`server/routes`): asocian cada URL y método HTTP con su controlador y sus middlewares.
- **Middlewares** (`server/middlewares`): funciones que se ejecutan antes del controlador, por ejemplo la verificación del token.
- **Servicios** (`server/services`): lógica reutilizable, como el envío de correos.

3.3 Estructura de carpetas

```
PAW_FRIEND/
├── client/                 # Frontend (React + Vite)
│   └── src/
│       ├── Auth/           # Login y registro
│       ├── components/     # Navbar, Sidebar, Layout, calendario, campanita...
```

```
│       ├── contexts/       # AuthContext y NotificationContext
│       ├── hooks/          # useLogin, useSignup
│       ├── pages/          # Pantallas (mascotas, inventario, usuarios...)
│       ├── App.jsx         # Definición de rutas
│       └── main.jsx        # Punto de entrada
├── server/                 # Backend (Node + Express)
│   ├── controllers/  models/  routes/  middlewares/  services/  utils/
│   ├── __tests__/          # Pruebas unitarias (Jest)
│   └── index.js            # Punto de entrada del servidor
├── selenium_tests/         # Pruebas end-to-end
├── k6_tests/               # Pruebas de carga
└── nmap_tests/            # Escaneo de seguridad
```

3.4 Punto de entrada del servidor (index.js)

El archivo `server/index.js` inicia el backend y se organiza en cinco bloques:

1. **Middlewares globales:** CORS para permitir peticiones desde el cliente (puerto 5173) y `express.json()` para leer el cuerpo de las peticiones en JSON.
2. **Rutas:** registra cada módulo bajo su prefijo, por ejemplo `/api/pets` o `/api/auth`.
3. **Conexión a MongoDB:** mediante `mongoose.connect(process.env.MONGO_URI)`; la dirección se lee del archivo `.env`.
4. **Manejador global de errores:** recibe cualquier error del sistema y responde con su código de estado y un mensaje uniforme (500 por defecto).
5. **Arranque:** `app.listen(PORT)` inicia el servidor en el puerto definido (9000 por defecto).

4. Módulos del sistema

4.1 Autenticación y usuarios

Permite registrarse, iniciar sesión y controlar qué puede hacer cada usuario.

- **Registro:** valida los campos, verifica que el correo no exista y encripta la contraseña con `bcrypt.hash(password, 12)`. Se permite un máximo de 3 administradores.
- **Inicio de sesión:** compara la contraseña con `bcrypt.compare`, verifica que el usuario esté activo y devuelve un token JWT válido por 90 días.
- **Protección de rutas:** el middleware `protect` lee el encabezado `Authorization: Bearer <token>`, lo verifica y carga el usuario; `authorize('admin')` restringe la ruta según el rol (respuestas 401 y 403).
- **Gestión de usuarios:** el administrador puede crear, editar, activar/desactivar y eliminar usuarios, y cada usuario puede configurar sus preferencias de notificación.
- **En el cliente:** `AuthContext` guarda el token en `localStorage`, comprueba su vencimiento y expone `login`, `logout` e `isAuthenticated`; `ProtectedRoute` redirige al login a quien no tiene sesión.

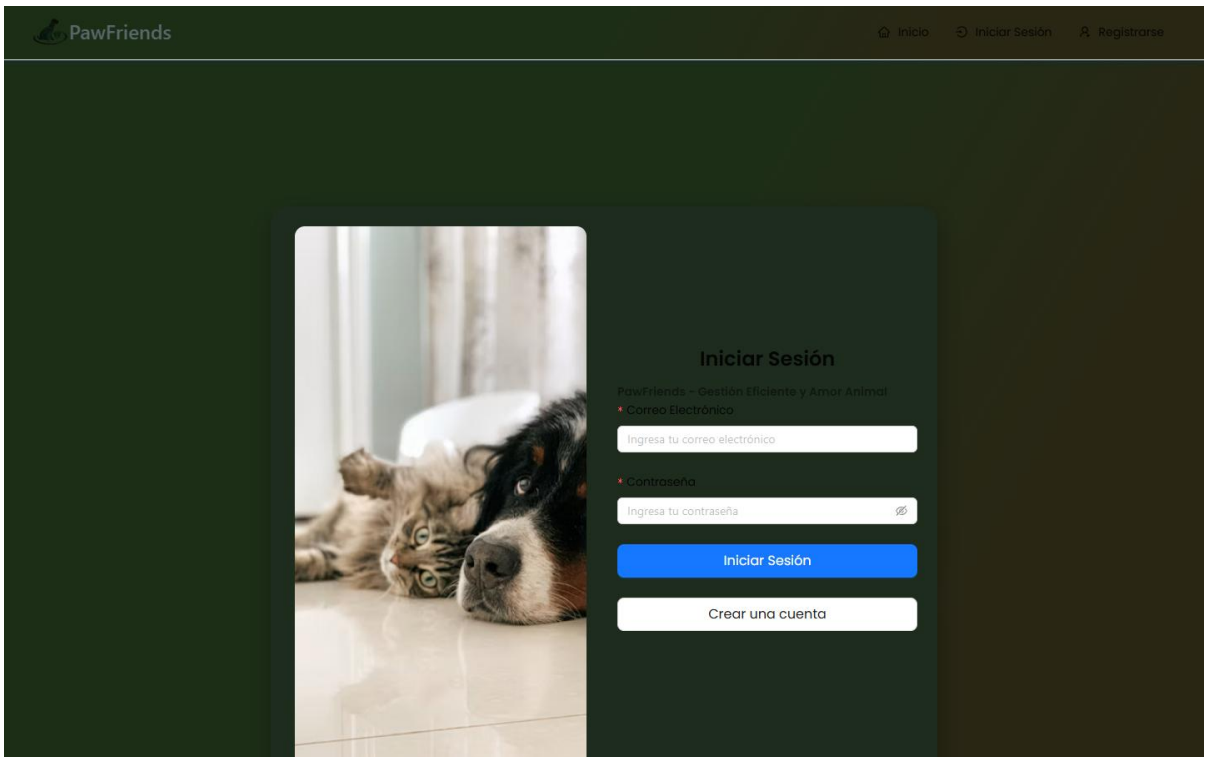


Figura 2. Pantalla de inicio de sesión.

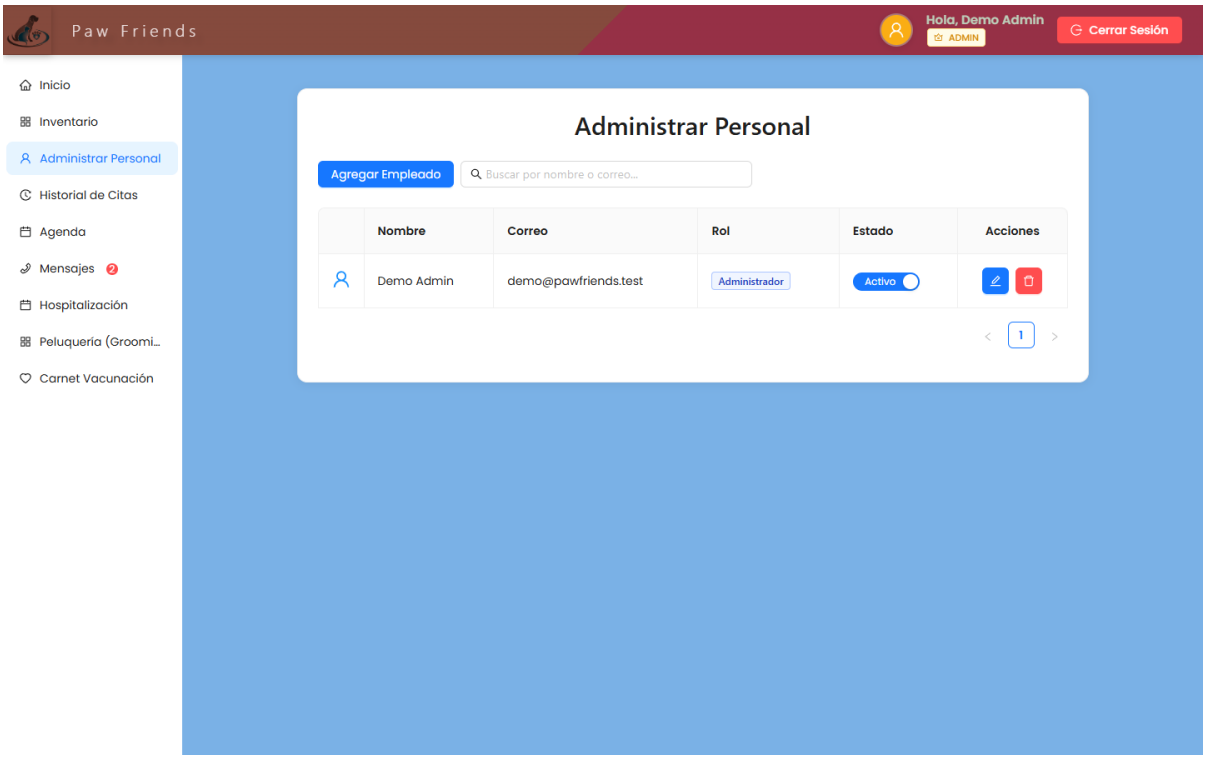


Figura 3. Administración del personal (solo administradores).

4.2 Mascotas

Registro completo de cada paciente: datos básicos (nombre, especie, raza, sexo, fecha de nacimiento, peso), datos del dueño, alergias, medicamentos, **historial médico** (consultas, cirugías, tratamientos, emergencias) y **vacunas** (vacuna, desparasitante, vitamina). Al registrar una mascota se notifica automáticamente a los administradores.

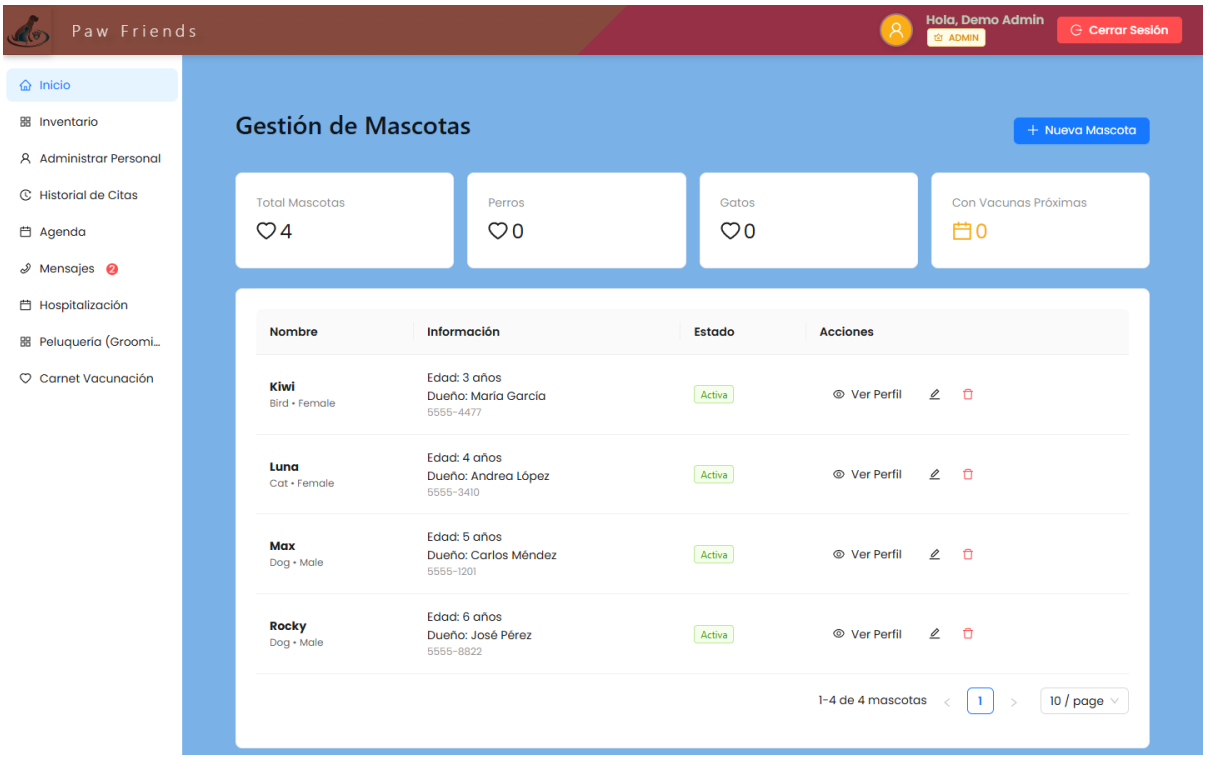


Figura 4. Gestión de mascotas.

4.3 Citas y agenda

Cada cita registra cliente, mascota, veterinario, fecha, duración, motivo, diagnóstico y tratamiento. Su estado puede ser programada, confirmada, en curso, completada, cancelada o no asistió. La API ofrece las citas del día, las próximas, las de cada veterinario y estadísticas. La agenda general muestra las citas sobre un calendario mensual.

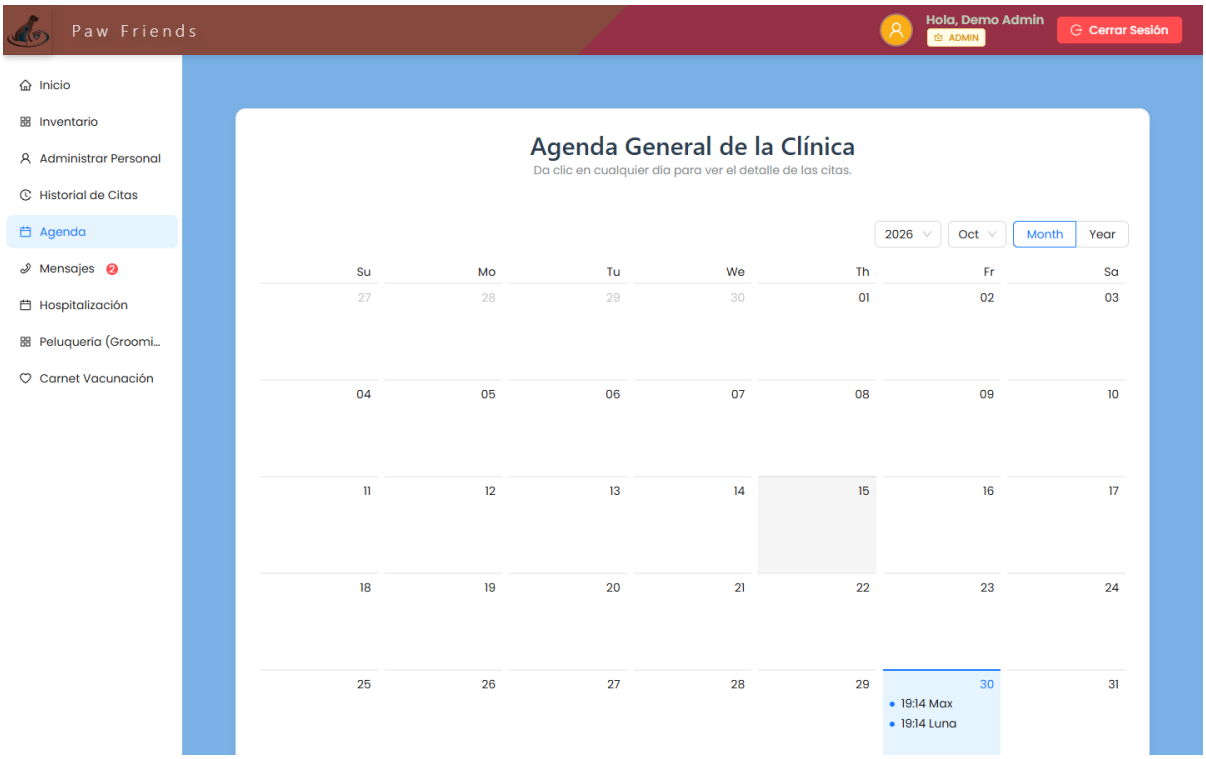


Figura 5. Agenda general de la clínica.

4.4 Inventario

CRUD de productos con nombre, descripción, tipo, categoría, precio, stock y fecha de ingreso. Al crear, editar o eliminar un producto se generan notificaciones, y si el stock es igual o menor a **5 unidades** se envía una alerta de **stock bajo** (urgente si quedan 2 o menos).

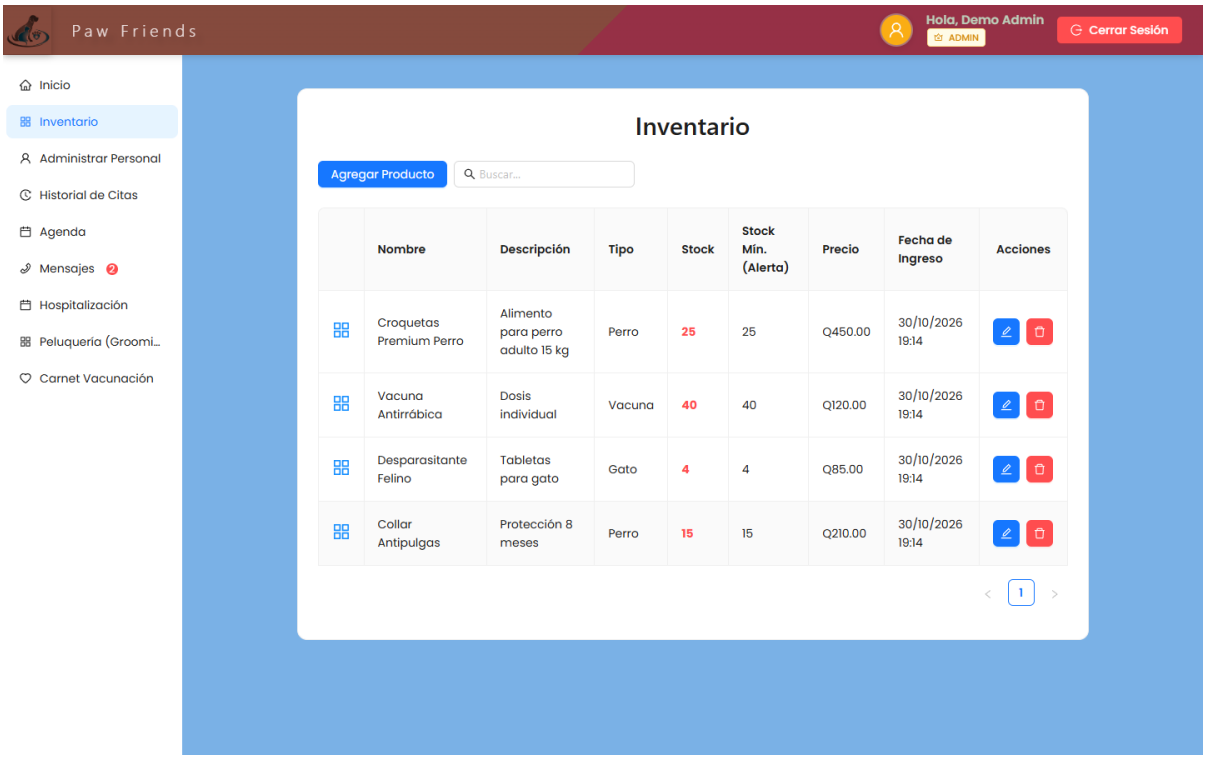


Figura 6. Inventario de productos.

4.5 Historial clínico

Registro de atenciones con mascota, dueño, contacto, medicamento, peso y estado (pendiente o completado). Permite buscar, editar y eliminar registros.

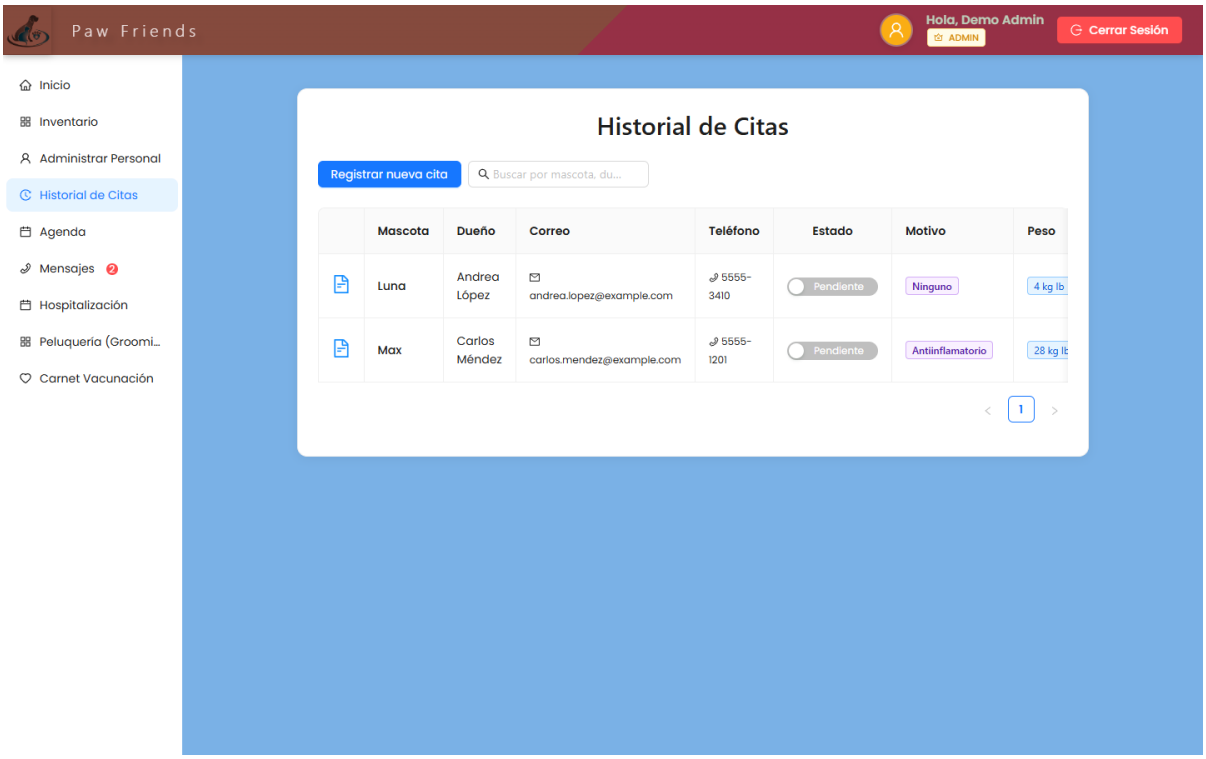


Figura 7. Historial de citas y atenciones.

4.6 Notificaciones

El sistema genera notificaciones de distintos tipos: stock bajo, producto nuevo/actualizado/eliminado, recordatorio médico, recordatorio de cita, alerta del sistema, contacto recibido y personalizadas, con prioridad baja, media, alta o urgente.

- **Tiempo real (Server-Sent Events):** el cliente abre una conexión permanente con `GET /api/notifications/events`; cuando ocurre un evento, el servidor envía la notificación al instante, sin recargar la página.
- **Correo electrónico:** el servicio `emailService` usa Nodemailer con plantillas HTML; si no hay credenciales configuradas, el envío se desactiva sin afectar al sistema.
- **Notificaciones de negocio:** el administrador puede crear recordatorios médicos, de citas, vacunación, desparasitación, seguimiento, cumpleaños de mascotas, productos por vencer y mantenimiento.
- **En el cliente:** `NotificationBell` muestra la campanita con el número de avisos sin leer.

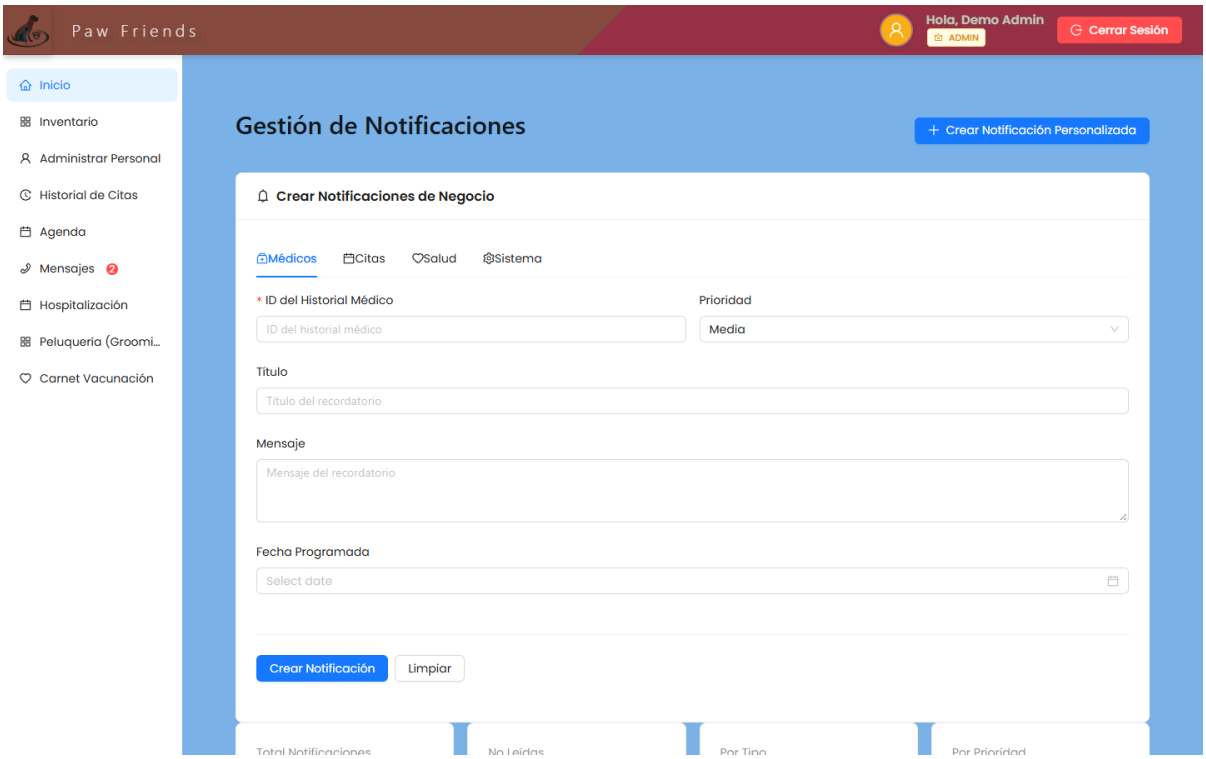


Figura 8. Gestión y creación de notificaciones.

4.7 Métricas y dashboard

El controlador de métricas calcula indicadores de citas, productos, mascotas y finanzas, consumidos por el dashboard ejecutivo y sus gráficas. La pantalla de inicio presenta un resumen con tarjetas de acceso rápido.

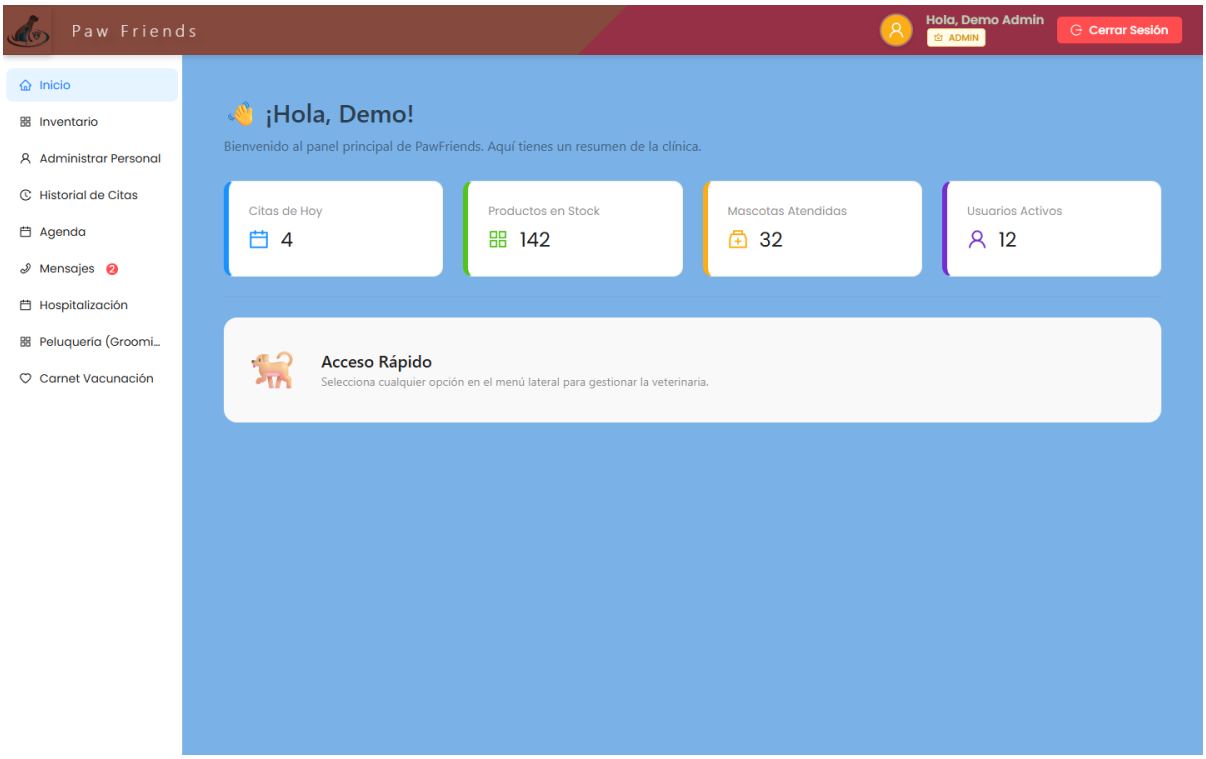


Figura 9. Panel principal (dashboard).

4.8 Contactos

Formulario público para que los clientes envíen consultas (nombre, apellido, correo, teléfono y descripción). El administrador los consulta en un directorio, puede marcarlos como “Pendiente” o “Atendido” y eliminarlos. El menú lateral muestra cuántos mensajes hay pendientes.

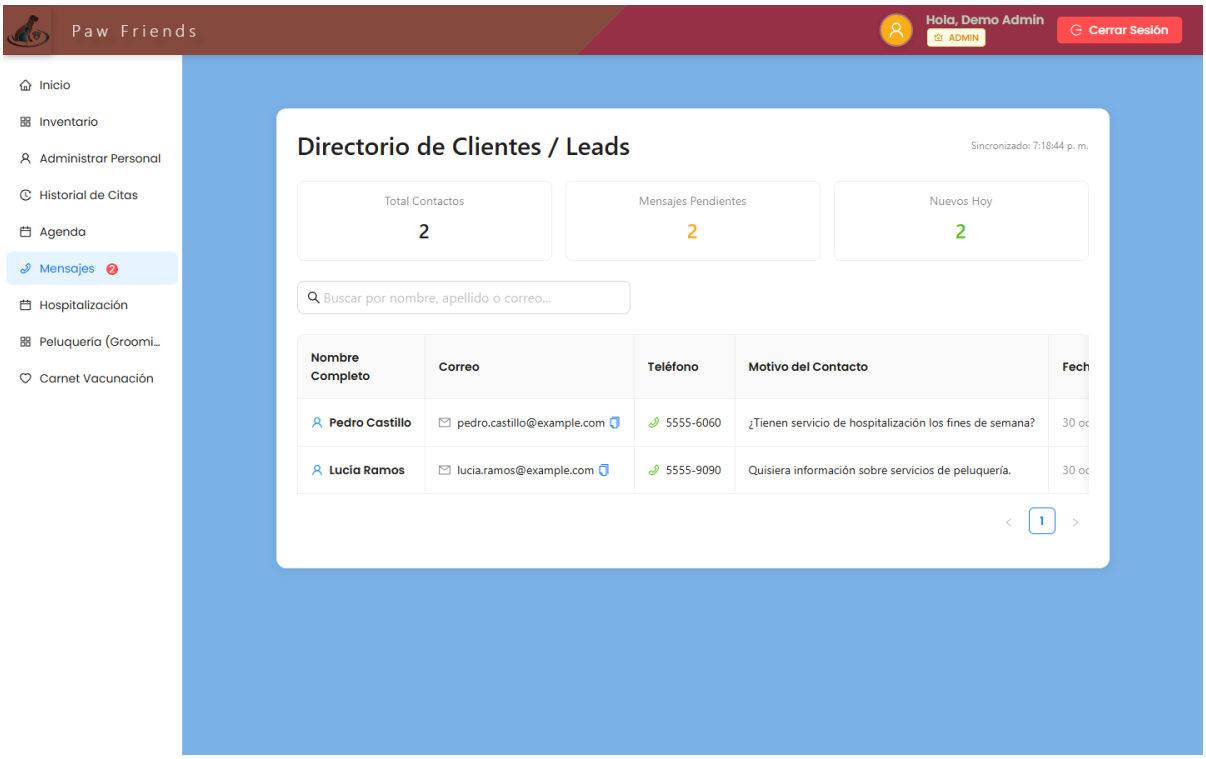


Figura 10. Directorio de contactos recibidos.

5. API REST

Todas las rutas tienen el prefijo `/api` y responden en JSON. En la columna “Acceso”: **Público** no requiere token, **Usuario** requiere sesión iniciada y **Admin** requiere rol de administrador.

Método	Ruta	Descripción	Acceso
POST	/api/auth/register	Registrar usuario	Público
POST	/api/auth/login	Iniciar sesión y obtener token	Público
GET	/api/auth/profile	Perfil del usuario autenticado	Usuario
GET / POST	/api/users	Listar / crear usuarios	Admin
PUT / DELETE	/api/users/:id	Editar / eliminar usuario	Admin
PATCH	/api/users/:id/estado	Activar o desactivar usuario	Admin
GET / PUT	/api/users/me/notifications/preferences	Preferencias de notificación	Usuario
GET	/api/pets	Listar mascotas	Usuario
POST / PUT / DELETE	/api/pets/:id	Crear, editar, eliminar mascota	Admin
POST	/api/pets/:id/medical-record	Agregar registro médico	Admin
POST	/api/pets/:id/vaccination	Agregar vacuna	Admin
GET	/api/appointments	Listar citas (filtros por fecha)	Usuario
GET	/api/appointments/today · /upcoming	Citas de hoy y próximas	Usuario
POST / PUT / DELETE	/api/appointments/:id	Crear, editar, eliminar cita	Admin
PUT	/api/appointments/:id/status	Cambiar estado de la cita	Admin
GET	/api/products	Listar productos	Público
POST	/api/products	Crear producto	Público
PUT / DELETE	/api/products/:id	Editar / eliminar producto	Admin
GET / POST	/api/history	Listar / crear historial	Público
PUT / DELETE	/api/history/:id	Editar / eliminar historial	Admin
GET	/api/notifications	Notificaciones del usuario	Usuario
GET	/api/notifications/events	Canal en tiempo real (SSE)	Usuario (token)
PUT	/api/notifications/:id/read	Marcar como leída	Usuario
POST	/api/business-notifications/...	Recordatorios de negocio	Admin
GET	/api/metrics/dashboard	Métricas generales	Usuario
POST / GET	/api/contacto	Enviar / listar contactos	Público

6. Base de datos

Se utiliza **MongoDB**, una base de datos NoSQL que almacena la información en documentos con formato similar a JSON. Cada colección se define con un esquema de Mongoose que establece los campos, tipos, valores permitidos y validaciones.

Colección	Campos principales
users	name, email (único), password (hash), role (user/admin/empleado), activo, notificationPreferences
pets	name, type, breed, gender, birthDate, weight, ownerName, ownerEmail, ownerPhone, allergies, medications, vaccinations[], medicalHistory[], isActive
appointments	title, clientName, clientEmail, petName, petType, veterinarian, appointmentDate, duration, status, services[], reason, diagnosis, treatment
products	name, description, price, stock, category, type, quantity, entryDate
histories	petName, ownerName, ownerEmail, ownerPhone, medication, weight, completed, date
notifications	title, message, type, priority, recipient (User), read, relatedProduct, relatedHistory
contacts	nombre, apellido, correo, numero, descripcion, estado (Pendiente/Atendido)

7. Seguridad

- **Contraseñas encriptadas:** se almacenan con bcrypt (factor de costo 12), nunca en texto plano.
- **Tokens JWT:** cada petición protegida debe incluir un token firmado con una clave secreta del servidor.
- **Control de acceso por roles:** las operaciones sensibles solo están disponibles para administradores.
- **Usuarios inactivos:** un usuario desactivado no puede iniciar sesión.
- **Variables de entorno:** credenciales, URI de la base de datos y clave JWT se guardan en `.env`, excluido del repositorio mediante `.gitignore`.
- **Manejo centralizado de errores:** las respuestas de error tienen un formato uniforme y no exponen detalles internos fuera del modo desarrollo.
- **Escaneo de puertos:** se utilizó nmap para revisar los servicios expuestos.

8. Pruebas y control de calidad

Tipo	Herramienta	Qué se probó	Resultado
Unitarias	Jest	Validaciones de usuarios, mascotas, citas, productos, notificaciones y controlador de autenticación	7 suites, 37 pruebas: todas aprobadas
End-to-end	Selenium	Login, registro, listado de usuarios, creación de mascota, cita y producto	Scripts en selenium_tests/
Carga	k6	Múltiples usuarios iniciando sesión de forma simultánea	Script en k6_tests/
Seguridad	nmap	Puertos abiertos y servicios expuestos	Script en nmap_tests/

Las pruebas unitarias se ejecutan con:

```
cd server
npm test
```

9. Instalación y ejecución

9.1 Requisitos

- Node.js 18 o superior y npm.
- Una base de datos MongoDB (local o en la nube con MongoDB Atlas).
- Git para clonar el repositorio.

9.2 Pasos

1. Clonar el repositorio: `git clone https://github.com/Edgarbarco/PAW_FRIEND.git`
2. Crear `server/.env` a partir de `server/.env.example` con `PORT`, `MONGO_URI`, `JWT_SECRET` y, opcionalmente, las credenciales de correo.
3. Crear `client/.env` con `VITE_API_URL=http://localhost:9000`.
4. Instalar y ejecutar el backend: `cd server`, `npm install`, `npm run dev`.
5. En otra terminal, instalar y ejecutar el frontend: `cd client`, `npm install`, `npm run dev`.
6. Abrir `http://localhost:5173` en el navegador.

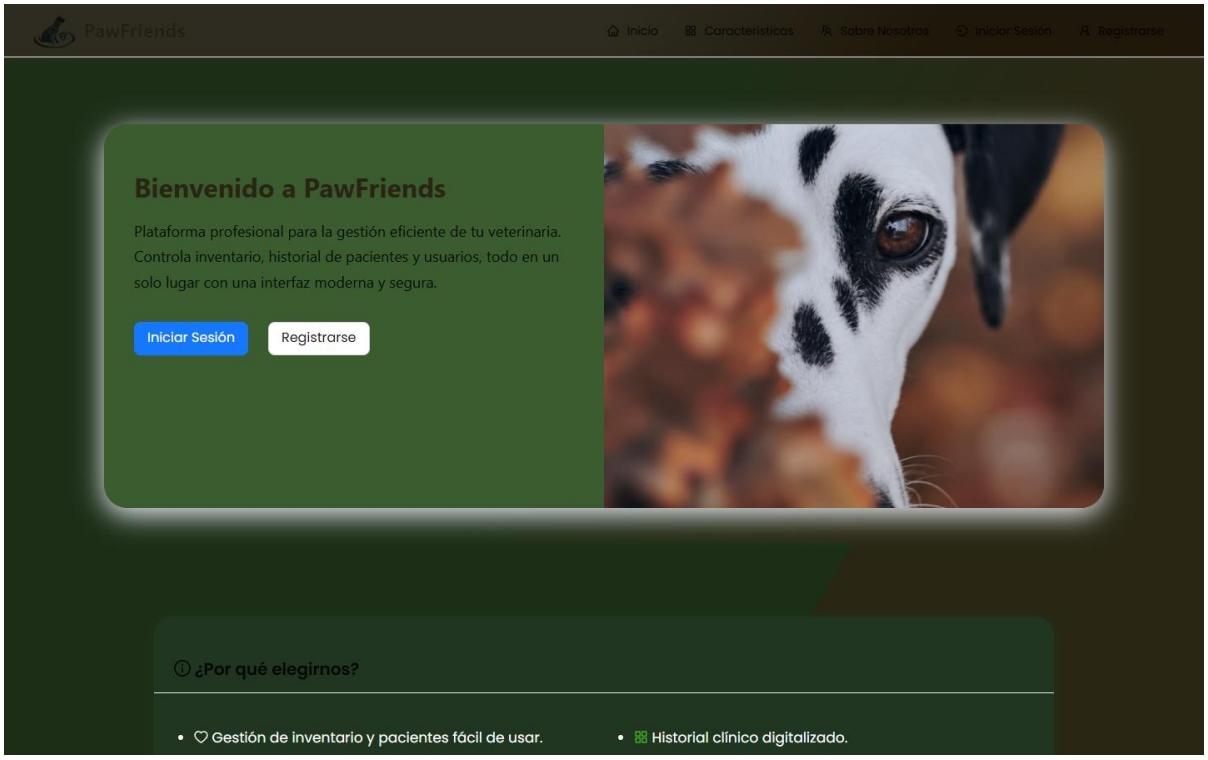


Figura 11. Página de inicio del sistema.

10. Distribución del trabajo

El proyecto se desarrolló de forma colaborativa en GitHub. La siguiente distribución se obtuvo del historial de commits del repositorio, por lo que cada aporte es verificable.

Integrante	Responsabilidad	Commits
Edgar Leonel Barco Cruz	Líder · Autenticación, usuarios, pantallas de servicios y diseño final	7
Alan Natanihel Ruano Corvalán	Configuración base del servidor y del cliente	3
Ergil Isaías Cárdenas Cruz	Estructura del frontend, login/registro, sesión y Jest	5
Enma Leticia Ramírez Aquino	Mascotas, citas, inventario y estructura visual	9
Humberto Herminio Duarte Aguilar	Notificaciones, métricas, historial, contactos, k6 y nmap	6
Julio César Sandoval Benítez	Dashboard, contactos e interfaz de notificaciones	3

10.1 Edgar Leonel Barco Cruz — Líder del proyecto

- Modelo de usuario, registro e inicio de sesión (`UserModel.js`, `authController.js`, `authRoute.js`).
- Middleware JWT y control por roles (`authmiddleware.js`), gestión de usuarios (`userController.js`, `userRoutes.js`).
- Pantallas de usuarios, agenda, vacunación, peluquería y hospitalización.
- Pruebas Selenium de login, registro y listado de usuarios.
- Ajustes de diseño final (Home, Login, Register, Navbar, Sidebar) e integración de ramas.

10.2 Alan Natanihel Ruano Corvalán — Configuración base

- Estructura inicial del repositorio, `README.md` y `.gitignore`.
- Servidor Express: `server/index.js` (middlewares, registro de rutas, conexión a MongoDB, manejador global de errores y arranque), `package.json`, `.env.example`.
- Clase `AppError` (`utils/appError.js`) para errores con formato uniforme.
- Configuración del cliente con Vite y ESLint (`vite.config.js`, `eslint.config.js`, `package.json`, `.env.example`).

10.3 Ergil Isaías Cárdenas Cruz — Estructura del frontend

- Punto de entrada y rutas del cliente (`main.jsx`, `App.jsx`, `App.css`).
- Pantallas de login y registro con sus hooks (`useLogin`, `useSignup`).
- Contextos de sesión y notificaciones (`AuthContext`, `NotificationContext`).

- Archivos base del servidor (modelos y controladores iniciales de usuarios, mascotas y notificaciones).
- Pruebas unitarias con Jest (`server/__tests__`, `jest.config.js`).

10.4 Enma Leticia Ramírez Aquino — Módulos principales

- CRUD de mascotas, citas y productos (modelos, controladores y rutas).
- Layout, Navbar, Sidebar, ProtectedRoute y página de inicio.
- Pantallas de mascotas, perfil de mascota, calendario de citas, inventario e historial.
- Pruebas Selenium de creación de mascota, cita y producto; correcciones de nombres de archivos e integración.

10.5 Humberto Herminio Duarte Aguilar — Notificaciones y métricas

- Modelo, controlador, rutas y middleware de notificaciones (incluye tiempo real con SSE y alerta de stock bajo).
- Servicios de correo (`emailService.js`) y notificaciones (`notificationService.js`).
- Métricas (`metricsModel`, `metricsController`, `metricsRoutes`).
- API de historial clínico y contactos.
- Pruebas de carga con k6 y escaneo de seguridad con nmap.

10.6 Julio César Sandoval Benítez — Dashboard y contactos

- Dashboard ejecutivo y gráficas (`ExecutiveDashboard`, `ChartsComponents`, `Dashboard`, `DashboardPage`).
- Formulario de contacto y administración de contactos (`ContactForm`, `ContactsAdmin`).
- Campanita y gestión de notificaciones, creador de notificaciones de negocio.
- Pruebas Selenium de navegación y contactos.

11. Conclusiones

- Se desarrolló un sistema web funcional que centraliza la gestión de una veterinaria: mascotas, citas, inventario, usuarios, notificaciones y métricas.
- El stack MERN permitió usar JavaScript en todo el proyecto y separar claramente el cliente del servidor, lo que facilitó el trabajo en paralelo del equipo.
- La organización MVC del backend y la estructura por componentes del frontend permitieron que cada integrante desarrollara su módulo de forma independiente y lo integrara sobre una misma base.
- La seguridad se abordó con contraseñas encriptadas, tokens JWT, roles y variables de entorno.
- Las pruebas unitarias, end-to-end, de carga y de seguridad aportaron confianza sobre el funcionamiento del sistema.

Anexo A. Capturas adicionales

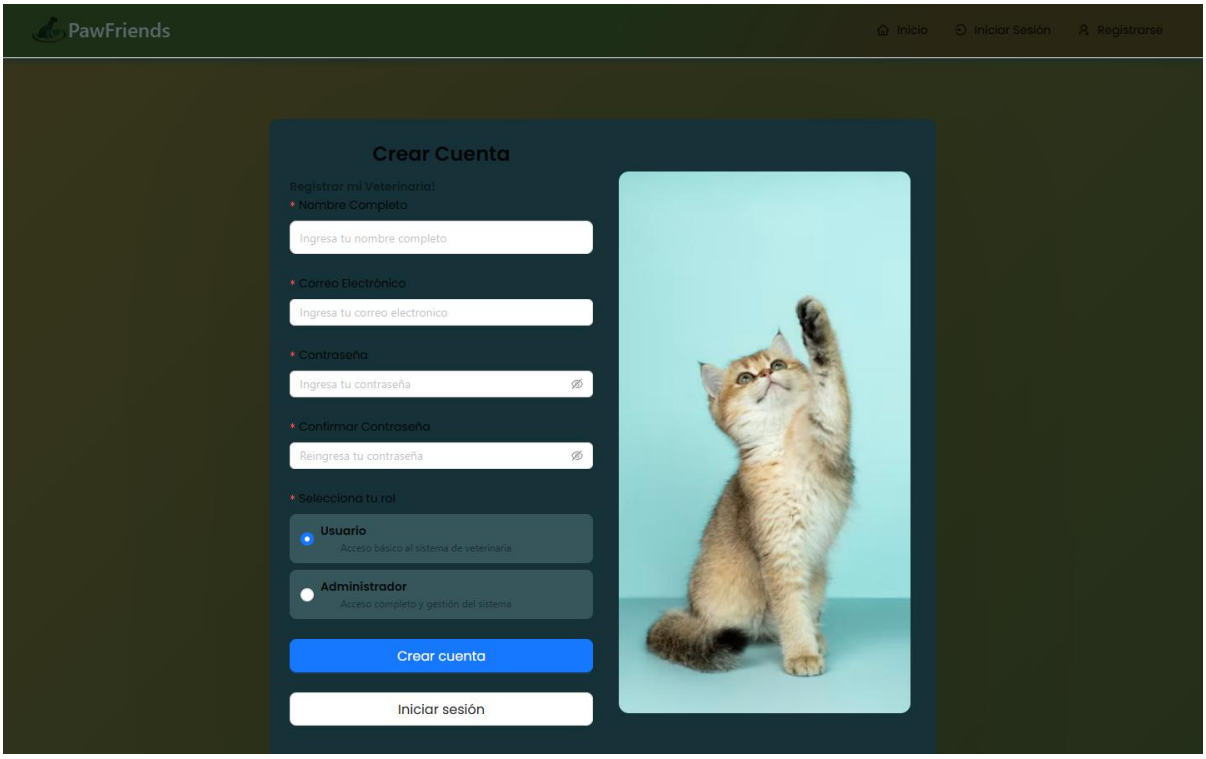


Figura 12. Pantalla de registro de usuarios.